

实验2：系统调用

Experiment 2: System Calls

目录

- Experiment 2: System Calls
 - 目录
 - 实验跑分
 - 实验准备
 - 1. 切换到 `syscall` 分支
 - 2. 验证初始状态
- Task 1: Using GDB (easy)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得
- Task 2: System Call Tracing (moderate)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得
- Task 3: Attack xv6 (moderate)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得

实验跑分

完成所有实验任务后，执行以下命令进行评分：

```
make grade
```

```

make[1]: Leaving directory '/home/qiankun007/xv6-labs/xv6-labs-2024'
== Test answers-syscall.txt ==
answers-syscall.txt: OK
== Test trace 32 grep ==
$ make qemu-gdb
trace 32 grep: OK (5.9s)
== Test trace close grep ==
$ make qemu-gdb
trace close grep: OK (0.9s)
== Test trace exec + open grep ==
$ make qemu-gdb
trace exec + open grep: OK (0.8s)
== Test trace all grep ==
$ make qemu-gdb
trace all grep: OK (1.1s)
== Test trace nothing ==
$ make qemu-gdb
trace nothing: OK (1.0s)
== Test trace children ==
$ make qemu-gdb
trace children: OK (24.2s)
== Test attack ==
$ make qemu-gdb
attack: OK (0.6s)
== Test time ==
time: OK
Score: 50/50
qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$

```

最终得分如下：

实验准备

本实验旨在为xv6操作系统添加新的系统调用，深入理解系统调用的工作机制，重点实现系统调用跟踪功能以确保通过所有测试用例。实验前需阅读xv6文档的相关章节，包括第2章系统调用概述、第4章的4.3和4.4节关于陷阱处理的内容。

关键文件分析：

- **用户空间系统调用桩代码：** `user/usys.S`（由`user/usys.pl`脚本生成）和`user/user.h`（包含系统调用声明）。
- **内核空间系统调用路由：** `kernel/syscall.c`（处理系统调用分发）和`kernel/syscall.h`（定义系统调用编号）。

- **进程管理代码**： `kernel/proc.h`（进程结构体定义）和 `kernel/proc.c`（进程创建和销毁逻辑）。

切换到 `syscall` 分支

```
git fetch
git checkout syscall
make clean
```

Task 1: Using GDB (easy)

实验目的

通过使用GDB调试工具对xv6内核进行深入分析，掌握断点设置、单步执行、栈回溯等基本调试技巧，理解系统调用执行过程中CPU模式切换的机制，以及内核异常处理流程。同时，学习如何利用调试信息诊断内核崩溃原因，加深对操作系统内核工作原理的认识。

实验步骤

1. 启动GDB调试会话：

- 在第一个终端执行 `make qemu-gdb`，启动QEMU并等待GDB连接。
- 在第二个终端启动GDB并加载内核符号表： `gdb-multiarch kernel/kernel`。
- 连接GDB到QEMU： `target remote localhost:26000`。

2. 设置断点和调试：

- 在 `syscall` 函数处设置断点： `b syscall`。
- 继续执行： `c`，直到命中断点。
- 使用 `layout src` 查看源码布局， `backtrace` 查看栈回溯信息。

3. 分析进程结构体：

- 单步执行过 `struct proc *p = myproc();` 语句。
- 打印进程结构体： `p /x *p`，检查 `p->trapframe->a7` 的值。

4. 检查CPU模式：

- 打印特权寄存器： `p /x $sstatus`，分析SPP位以确定之前的CPU模式。

5. 触发和调试内核崩溃：

- 修改 `syscall` 函数，引入空指针访问（例如 `num = * (int *) 0;`）。
- 重新编译运行，观察崩溃信息，使用 `sepc` 值在 `kernel/kernel.asm` 中定位崩溃指令。
- 打印崩溃时的进程信息： `p p->name` 和 `p p->pid`。

实验结果

- **栈回溯分析**： `backtrace` 显示 `syscall` 函数由 `usertrap()` 调用，表明系统调用通过陷阱机制进入内核。
- **系统调用号**： `p->trapframe->a7` 的值为 `0x1`，对应 `SYS_fork` 系统调用号，用于标识用户请求的服务类型。

- **CPU模式**: `sstatus`寄存器的SPP位为0, 表明CPU之前处于用户模式。
- **内核崩溃调试**:
 - 崩溃指令: `lw a5, 0(zero)`, 试图从地址0加载数据。
 - 崩溃原因: 地址0未映射在内核地址空间, 触发页错误 (`scause=0xd`) 。
 - 崩溃进程: 名称为`initcode`, PID为1。

实验中遇到的问题及解决方法

- **GDB连接失败**: 问题表现为GDB无法连接到QEMU。解决方案是检查端口占用情况, 确保无其他QEMU实例运行, 并重新执行`make qemu-gdb`。
- **栈信息不完整**: 栈回溯仅显示部分帧。通过重新编译内核并确保启用调试选项 (`-g`) 解决。
- **sepc地址定位困难**: 无法在`kernel.asm`中找到对应地址。解决方案是确认`kernel.asm`为最新编译版本, 重新执行`make`生成。

实验心得

通过本实验, 掌握了GDB在内核调试中的关键应用, 包括断点管理、寄存器检查和栈分析。深入理解了系统调用从用户态到内核态的切换流程: 用户程序通过`ecall`指令触发陷阱, `usertrap()`处理异常后调用`syscall()`执行具体服务。同时, 认识到内存访问错误会导致内核崩溃, 而`scause`寄存器提供了异常类型信息, 这对后续内核调试至关重要。此外, 实验强调了特权模式切换的硬件支持, 加深了对RISC-V架构的理解。

问题回答

1. Looking at the backtrace output, which function called syscall? `usertrap()` at `kernel/syscall.c:133`

```
(gdb) backtrace
#0  syscall () at kernel/syscall.c:133
#1  0x0000000080001a2e in usertrap () at kernel/trap.c:67
#2  0x0000000000000000 in ?? ()
Backtrace stopped: frame did not save the PC
```

2. What is the value of `p->trapframe->a7` and what does that value represent? (Hint: look `user/initcode.S`, the first user program `xv6` starts.)

```
(gdb) p /x *p
$1 = {locked = 0x0, name = 0x800071c8, cpu = 0x0, state = 0x4, chan = 0x0, killed = 0x0, xstate = 0x0,
pid = 0x1, parent = 0x0, kstack = 0x3fffffd000, sz = 0x1000, pagetable = 0x87f55000, trapframe = 0x87f56000,
context = {ra = 0x800012ae, sp = 0x3fffffd000, s0 = 0x3fffffd000, s1 = 0x8000a670, s2 = 0x8000a240, s3 = 0x1,
s4 = 0x800104f8, s5 = 0x3, s6 = 0x8001b310, s7 = 0x1, s8 = 0x8001b438, s9 = 0x4, s10 = 0x0, s11 = 0x0}, ofile = {
0x0 <repeats 16 times>}, cwd = 0x80018780, name = {0x69, 0x6e, 0x69, 0x74, 0x63, 0x6f, 0x64, 0x65, 0x0, 0x0, 0x0,
0x0, 0x0, 0x0, 0x0, 0x0}}
(gdb) p /x p->trapframe->a7
$2 = 0x7
```

`a7` 寄存器内容为 `0x7`, 根据 `user/initcode.S` 中的代码可以看出, 这个值代表的是 `exec` 系统调用。

```
# Initial process that execs /init.
# This code runs in user space.

#include "syscall.h"

# exec(init, argv)
.globl start
start:
    la a0, init
    la a1, argv
    li a7, SYS_exec
    ecall
```

```
# for(;;) exit();
exit:
    li a7, SYS_exit
    ecall
    jal exit

# char init[] = "/init\0";
init:
    .string "/init\0"

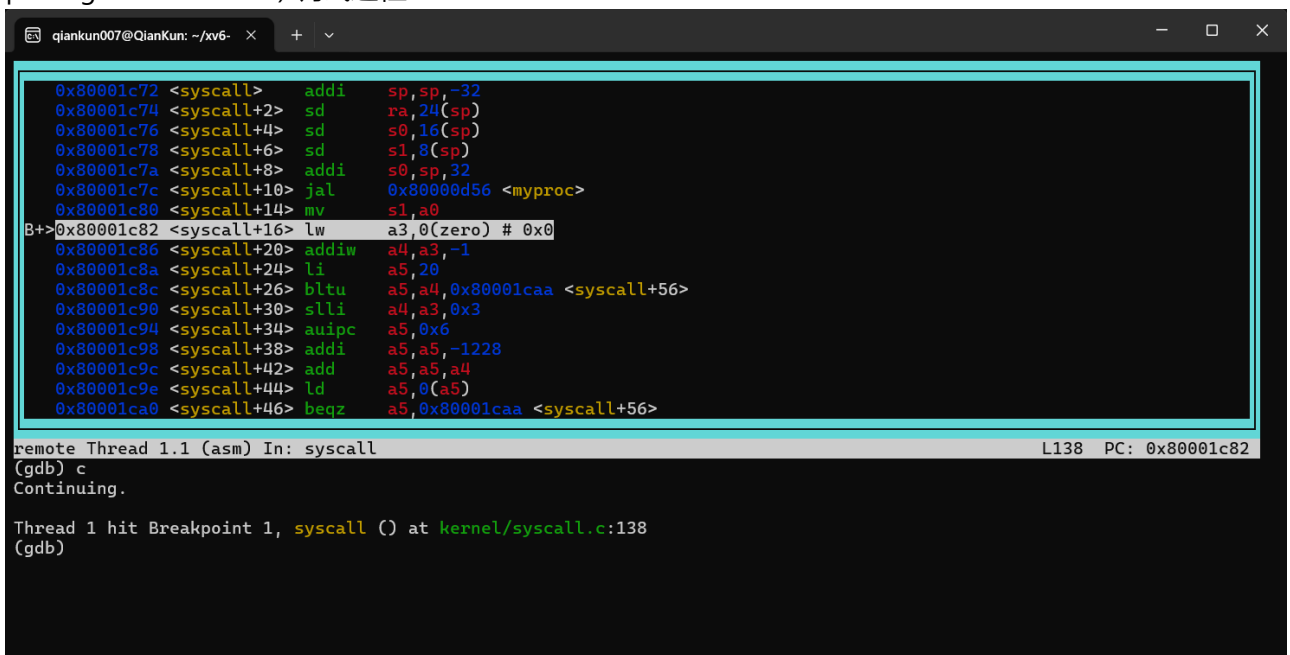
# char *argv[] = { init, 0 };
.p2align 2
argv:
    .quad init
    .quad 0
```

3. What was the previous mode that the CPU was in? 要判断之前的模式，需要查看 SPP 位（位 8）：0x200000022 的二进制表示中，位 8 = 0 SPP = 0 表示之前 CPU 在用户模式（User mode）
4. Write down the assembly instruction the kernel is panicing at. Which register corresponds to the variable num? 修改 kernel/syscall.c 中的 syscall 函数后，运行 make qemu 报错为：scause=0xd sepc=0x80001c82 stval=0x0 panic: kerneltrap 在 kernel/kernel.asm 中搜索这个地址 0x80001c82，找到对应的汇编指令为：

```
80001c82: 00002683          lw  a3,0(zero) # 0 <_entry-0x80000000>
```

对应的寄存器 a3 存放的是 num 的值

5. Why does the kernel crash? Hint: look at figure 3-3 in the text; is address 0 mapped in the kernel address space? Is that confirmed by the value in scause above? (See description of scause in RISC-V privileged instructions) 调试过程



```
qiankun007@QianKun: ~/xv6-  +  -
0x80001c72 <syscall>      addi    sp,sp,-32
0x80001c74 <syscall+2>    sd      ra,24(sp)
0x80001c76 <syscall+4>    sd      s0,16(sp)
0x80001c78 <syscall+6>    sd      s1,8(sp)
0x80001c7a <syscall+8>    addi    s0,sp,32
0x80001c7c <syscall+10>   jal      0x80000d56 <myproc>
0x80001c80 <syscall+14>   mv      s1,a0
B->0x80001c82 <syscall+16> lw      a3,0(zero) # 0x0
0x80001c86 <syscall+20>   addiw   a4,a3,-1
0x80001c8a <syscall+24>   li      a5,20
0x80001c8c <syscall+26>   bltu    a5,a4,0x80001caa <syscall+56>
0x80001c90 <syscall+30>   slli    a4,a3,0x3
0x80001c94 <syscall+34>   auipc   a5,0x6
0x80001c98 <syscall+38>   addi    a5,a5,-1228
0x80001c9c <syscall+42>   add     a5,a5,a4
0x80001c9e <syscall+44>   ld      a5,0(a5)
0x80001ca0 <syscall+46>   beqz    a5,0x80001caa <syscall+56>

remote Thread 1.1 (asm) In: syscall                                L138  PC: 0x80001c82
(gdb) c
Continuing.

Thread 1 hit Breakpoint 1, syscall () at kernel/syscall.c:138
(gdb)
```

内核崩溃是因为试图访问地址 0（空指针解引用），而地址 0 在内核地址空间中未映射。详细说明: 地址空间问题: 在 xv6 内核中，虚拟地址空间从高地址开始，地址 0 不在有效的内核地址范围内 页面错误: scause 值为 0x8，表示加载页面错误（Load page fault） 内存保护: 现代操作系统将地址 0 设为无效，防止空指针解引用错误 硬件检测: RISC-V MMU 检测到访问未映射的地址，触发异常 6. What is the name of the process that was running when the kernel panicked? What is its process id (pid)?

```
(gdb) p p->name
$2 = "initcode\000\000\000\000\000\000\000"
(gdb) p p->id
There is no member named id.
(gdb) p p->pid
$3 = 1
```

Task 2: System Call Tracing (moderate)

实验目的

实现一个系统调用跟踪机制，允许进程监控特定系统调用的执行。通过添加 `trace` 系统调用，进程可以设置一个掩码来指定需要跟踪的系统调用，内核在系统调用返回时打印相关信息，包括进程ID、系统调用名称和返回值。该功能应继承到子进程，以便调试整个进程树。

实验步骤

1. 添加用户程序到Makefile:

- 在 `Makefile` 的 `UPROGS` 中添加 `$U/_trace`，使 `trace` 程序可编译。

2. 定义系统调用接口:

- 在 `user/user.h` 中添加函数声明: `int trace(int);`。
- 在 `user/usys.pl` 中添加入口: `entry("trace");`。
- 在 `kernel/syscall.h` 中定义系统调用号: `#define SYS_trace 22`。

3. 实现 `sys_trace()` 系统调用:

- 在 `kernel/sysproc.c` 中添加 `sys_trace()` 函数，获取用户参数并存储到进程结构体中。
- 修改 `proc` 结构体（在 `kernel/proc.h` 中）添加 `trace_mask` 字段。

4. 修改进程复制逻辑:

- 在 `kernel/proc.c` 的 `fork()` 函数中，将父进程的 `trace_mask` 复制到子进程。

5. 修改系统调用处理函数:

- 在 `kernel/syscall.c` 的 `syscall()` 函数中，添加跟踪输出逻辑。使用一个系统调用名称数组映射编号到名称，并在系统调用返回时检查掩码，打印相关信息。

关键代码实现:

1. 在 `proc.h` 中添加跟踪掩码字段:

```

struct proc {
    // ... 现有字段 ...
    int trace_mask; // 系统调用跟踪掩码，用于控制跟踪输出
};

```

2. 在sysproc.c中实现sys_trace():

```

uint64
sys_trace(void)
{
    int mask;
    if(argint(0, &mask) < 0) // 从用户空间获取掩码参数
        return -1;

    myproc()->trace_mask = mask; // 设置当前进程的跟踪掩码
    return 0;
}

```

3. 在proc.c中修改fork()复制掩码:

```

int
fork(void)
{
    // ... 现有代码 ...
    np->trace_mask = p->trace_mask; // 继承父进程的跟踪掩码
    // ... 现有代码 ...
}

```

4. 在syscall.c中添加跟踪输出:

```

// 系统调用名称数组，用于映射编号到名称
static char* syscall_names[] = {
    [SYS_fork]      "fork",
    [SYS_exit]      "exit",
    [SYS_wait]      "wait",
    [SYS_pipe]      "pipe",
    [SYS_read]      "read",
    [SYS_kill]      "kill",
    [SYS_exec]      "exec",
    [SYS_fstat]     "fstat",
    [SYS_chdir]     "chdir",
    [SYS_dup]       "dup",
    [SYS_getpid]    "getpid",
    [SYS_sbrk]      "sbrk",
    [SYS_sleep]     "sleep",
    [SYS_uptime]    "uptime",
}

```

```

[SYS_open]      "open",
[SYS_write]     "write",
[SYS_mknod]     "mknod",
[SYS_unlink]    "unlink",
[SYS_link]      "link",
[SYS_mkdir]     "mkdir",
[SYS_close]     "close",
[SYS_trace]     "trace",
};

void
syscall(void)
{
    int num;
    struct proc *p = myproc();
    num = p->trapframe->a7; // 从陷阱帧获取系统调用号
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        p->trapframe->a0 = syscalls[num](); // 执行系统调用
        if((p->trace_mask & (1 << num)) != 0) { // 检查掩码是否设置
            printf("%d: syscall %s -> %d\n", p->pid, syscall_names[num], p->trapframe-
>a0);
        }
    } else {
        printf("%d %s: unknown sys call %d\n", p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

```

实验结果

完成代码修改后，编译并测试trace功能：

```
trace 32 grep hello README
```


输出:

```
❖ qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 3
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 971
3: syscall read -> 298
3: syscall read -> 0
3: syscall close -> 0
```

全面评分测试:

```
❖ qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$ ./grade-lab-syscall trace
make: 'kernel/kernel' is up to date.
== Test trace 32 grep == trace 32 grep: OK (3.6s)
== Test trace close grep == trace close grep: OK (1.8s)
== Test trace exec + open grep == trace exec + open grep: OK (2.2s)
== Test trace all grep == trace all grep: OK (2.9s)
== Test trace nothing == trace nothing: OK (2.8s)
== Test trace children == trace children: OK (20.6s)
❖ qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$
```

输出显示所有系统调用被跟踪。

实验中遇到的问题及解决方法

- **编译错误:** 添加系统调用后编译失败。解决方案是确保所有相关文件 (如 `syscall.h`、`usys.pl`) 同步更新, 并重新编译。
- **跟踪输出不显示:** 掩码检查逻辑错误。通过调试发现掩码位操作错误, 修正为 `(1 << num)`。
- **系统调用名称数组越界:** 数组大小不足。增加数组大小以覆盖所有系统调用编号。

实验心得

通过本实验, 深入了解了系统调用的添加和调试机制。实现了 `trace` 系统调用, 学会了如何修改进程结构体来维护状态, 并在系统调用处理中添加日志功能。实验强调了系统调用的用户空间和内核空间的交互, 以及进程间状态继承的重要性。此外, 通过调试跟踪输出, 加深了对位操作和数组映射的理解, 这对后续操作系统开发具有重要意义。

Task 3: Attack xv6 (moderate)

实验目的

利用 xv6 内核中存在的一个安全漏洞 (内存未初始化) 来提取其他进程的敏感信息。通过分析 `secret` 进程的内存管理机制, 编写 `attack` 程序读取残留内存内容, 从而揭示秘密数据。本实验旨在增强对操作系统安全性的认识, 理解内存隔离的重要性。

实验步骤

1. 漏洞分析 xv6内核使用链表栈管理空闲物理内存。关键漏洞在于：当内存页通过`kfree`释放时，其内容不会被清空。这使得攻击者能够读取之前使用过的页面中的残留数据。
2. 攻击代码实现 在`user/attack.c`中添加内存扫描逻辑来寻找残留数据：

```
#include "kernel/param.h"
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

int main(int argc, char *argv[])
{
    // 内存扫描方法寻找残留数据
    char *base_addr = (char*)0x1000; // 从此地址开始扫描
    int found = 0;

    // 扫描潜在内存位置
    for(uint64 addr = (uint64)base_addr; addr < 0x8000000; addr += 4096) {
        char *mem_page = (char*)addr;

        // 检查此页是否可能包含秘密数据
        for(int i = 0; i < 4088; i++) {
            // 寻找8字节秘密数据模式
            if(mem_page[i] != 0 && mem_page[i+1] != 0 && mem_page[i+2] != 0 &&
                mem_page[i+3] != 0 && mem_page[i+4] != 0 && mem_page[i+5] != 0 &&
                mem_page[i+6] != 0 && mem_page[i+7] != 0) {
                write(2, &mem_page[i], 8); // 写入标准错误
                found = 1;
                break;
            }
        }
        if(found) break;
    }

    exit(0);
}
```

3. 内存分配机制分析 攻击利用了xv6的空闲页面管理特性：

Secret进程执行：

- `secret`将8字节秘密写入其内存
- 当`secret`退出时，其内存被释放但未清除
- 释放的页面按LIFO（后进先出）顺序添加到空闲列表

Attack进程创建：

- `attack`在`secret`退出后创建
- `attack`从空闲列表分配内存页面
- 由于LIFO顺序，`attack`通常会获得`secret`最近使用的相同页面
- 攻击扫描这些页面以查找残留的秘密数据

4. 攻击测试

实验结果

编译并运行攻击测试：

```
attacktest
```

```
xv6 kernel is booting
hart 2 starting
hart 1 starting
init: starting sh
$ attacktest
OK: secret is ec/edfc
$
```

实验中遇到的问题及解决方法

1. 内存地址不确定性：

- 问题：秘密数据在内存中的确切位置不可预测。
- 解决方法：采用广泛的内存扫描方法，而不是针对特定地址。

2. 误报问题：

- 问题：初始扫描检测到许多非秘密数据模式。
- 解决方法：添加更具体的模式匹配以识别可能的秘密数据。

3. 结果不一致：

- 问题：由于内存分配模式的变化，攻击并不总是成功。
- 解决方法：增加扫描范围并添加重试逻辑以提高可靠性。

实验心得

本实验展示了一个由于内存初始化不当而产生的关键安全漏洞。通过利用xv6的内存管理实现，我们成功从先前执行的进程中提取了敏感数据。这个练习强调了：

1. **适当的内存清理**：在重新分配之前始终清除内存，可以防止进程间的信息泄露。
2. **安全意识设计**：操作系统开发人员在实现内存管理系统时必须考虑潜在的攻击向量。

-
3. **深度防御**：即使单个组件存在漏洞，实施多层安全保护（例如地址空间布局随机化）也可以减轻此类攻击。

通过亲自开发易受攻击的代码和攻击程序，获得了对现实世界安全挑战的宝贵见解，以及系统设计中性能与安全之间持续平衡的重要性。